

MASTER OF COMPUTER APPLICATIONS (MCA) — PROJECT SYNOPSIS

Academic Year 2025–2026 · Final Year Capstone Project

YouTube Trend Intelligence: Near-Real-Time Video Analytics & Empirical Growth Intelligence Platform

Full-Stack Analytics · PostgreSQL Window Functions · YouTube Data API v3 · Next.js 14

Project Domain: Big Data Analytics & Web Intelligence

Architecture: Append-Only Temporal Storage

Backend & Database: Next.js App Router, PostgreSQL 18

Frontend Framework: React 18, Tailwind CSS, Recharts

1. INTRODUCTION

In digital media consumption, video platforms like YouTube serve billions of daily interactions. However, native platform analytics predominantly report static, cumulative counters (such as total views and likes) without revealing the underlying temporal dynamics of content propagation. Content strategists, market analysts, and academic researchers require granular visibility into *how fast* content is growing, audience engagement trajectories, and why particular media items ascend the trending chart.

YouTube Trend Intelligence (TrendX) is an advanced, production-grade video analytics platform designed to continuously monitor, capture, and compute near-real-time performance metrics for trending videos. The platform operates on a strict paradigm of **empirical observation**: it captures unmutated temporal snapshots into PostgreSQL, evaluates historical deltas via SQL window functions (`LAG()`), and calculates a transparent, multi-factor **Custom Trend Score** without relying on opaque machine-learning speculation.

2. PROBLEM STATEMENT

Existing analytics solutions and public video dashboards exhibit three fundamental deficiencies:

- Absence of Temporal Granularity:** Standard APIs expose only current absolute numbers. When a video reaches 5,000,000 views, there is no native record indicating whether those views accumulated steadily over 3 weeks or surged in the past 60 minutes.
- Vulnerability to Viral Outliers:** Simple weighted averages fail because view metrics (hundreds of thousands) and engagement ratios (e.g., 3% to 8%) operate on vastly disparate mathematical scales, causing raw view counts to overpower all other factors.
- API Quota Depletion Risks:** Careless polling and use of heavy search endpoints (such as `search.list` at 100 quota units per call) rapidly exhaust Google's 10,000 daily free allowance,

causing application failures.

3. PROJECT OBJECTIVES

- **Automated Periodic Ingestion:** Ingest the top 50 popular video records across multiple countries (India, United States, United Kingdom, Canada, Australia) on a 15-minute interval at a minimal cost of 1 quota unit per country.
- **Immutable Append-Only Snapshot Storage:** Maintain historical auditability in PostgreSQL by appending snapshot rows with compound unique constraints (`video_id`, `region`, `captured_at`) rather than overwriting current records.
- **Temporal Rate Calculation:** Dynamically compute observed views gained, window elapsed time, and **view velocity (views gained per hour)** using the SQL `LAG()` window function.
- **Transparent Custom Trend Score:** Implement a percentile-normalized multi-factor scoring engine using `PERCENT_RANK()` across View Velocity (50%), Engagement Rate (20%), Publication Recency (15%), and Absolute Reach (15%).
- **Deterministic Rule-Based Rising Detection:** Identify organic viral breakouts using verified thresholds ($\geq 10,000$ view floor and $\geq 5,000$ views/hr velocity), preventing trivial low-view anomalies from distorting the chart.
- **Interactive Visual Intelligence Dashboard:** Provide high-density charts, video dossiers, embedded playback, search/filtering, and CSV export capabilities.

4. SYSTEM ARCHITECTURE & DATA PIPELINE

The system follows a decoupling between data ingestion, analytical aggregation, and presentation layers:

```
YouTube Data API v3 (videos.list chart=mostPopular - 1 unit) | ▼ POST
/api/collect (Protected by COLLECT_SECRET bearer token) | ▼ PostgreSQL
Relational Storage (Append-Only Snapshot Pattern) |— channels (creator identity
& metadata) |— categories (regional taxonomy mappings) |— videos (immutable
video metadata: duration, published date) |— video_snapshots (APPEND-ONLY:
view_count, likes, comments, timestamp) |— collection_runs (audit logging:
quota units, execution status) | ▼ SQL Analytics Engine (src/lib/queries.ts) |—
LAG() & DISTINCT ON: Compute view_delta & views_per_hour |— PERCENT_RANK():
Multi-factor percentile normalization | ──────────── ▼ ▼
Next.js REST API Routes React Server Components (RSC) (/api/trending,
/api/rising, (Zero network hop direct query layer, /api/analytics, /api/video)
interactive Recharts dark theme dashboard)
```

5. DATABASE SCHEMA DESIGN

The PostgreSQL schema enforces strict referential integrity and optimizes temporal queries through targeted composite indexes:

Table Name	Primary Key / Constraint	Purpose & Design Decisions
channels	channel_id	Stores creator identity, channel name, and avatar URL. Created upon first appearance.
categories	(category_id, region)	Caches YouTube category taxonomies per country. Prevents repeated quota consumption.
videos	video_id	Stores static facts (title, duration, publish timestamp). View counts are omitted by design.
video_snapshots	id (BIGSERIAL) UNIQUE(video_id, region, captured_at)	Append-only temporal core. Every reading records view_count, like_count, comment_count. Prevents duplicate batch insertion.
collection_runs	id (BIGSERIAL)	Operational audit log tracking run start/end, videos processed, quota consumed, and status.

6. MATHEMATICAL FORMULATION & SCORING ENGINE

A. Observed Velocity & Growth Derivation

Using PostgreSQL window functions, each video's latest snapshot (S_t) is compared against its immediate predecessor (S_{t-1}):

```
View Delta ( $\Delta V$ ) = GREATEST( $V_{latest} - V_{previous}$ , 0) Elapsed Hours ( $\Delta t$ ) =
(captured_at_latest - captured_at_previous) in seconds / 3600.0 Velocity (VPH) =
 $\Delta V / \Delta t$  (Views gained per hour) Growth Percentage =  $((V_{latest} - V_{previous}) / V_{previous}) \times 100$ 
Engagement Rate =  $((Likes + Comments) / Views) \times 100$ 
```

B. Custom Trend Score Formulation

To eliminate scale distortion, each metric is converted into a percentile rank ($r \in [0.0, 1.0]$) across all currently charted items:

```
Trend Score = ROUND [ ( 0.50  $\times$   $r_{velocity}$  + 0.20  $\times$   $r_{engagement}$  + 0.15  $\times$ 
 $r_{recency}$  + 0.15  $\times$   $r_{popularity}$  )  $\times$  100 ]
```

- **View Velocity (50%):** Dominant indicator reflecting immediate viral acceleration.
- **Engagement Rate (20%):** Ratio of active participatory feedback (likes + comments) to passive impressions.
- **Publication Recency (15%):** Hours elapsed since upload (inverted so newer videos receive higher rank).
- **Popularity Reach (15%):** Absolute cumulative viewership establishing baseline reach.

7. FUNCTIONAL MODULES & IMPLEMENTATION

Module Name	Key Technologies	Functional Responsibility
1. Ingestion Engine	TypeScript, Fetch API	Communicates with YouTube Data API v3; parses ISO-8601 durations; sanitizes null statistics.
2. Collector Endpoint	Next.js Route Handlers	POST /api/collect protected by COLLECT_SECRET bearer token; handles batch database transactions.
3. Analytical Query Core	node-postgres (pg), SQL CTEs	Executes multi-stage CTEs with PERCENT_RANK(), DISTINCT ON, and LAG() window aggregations.
4. Analytics UI Dashboard	React Server Components, Tailwind CSS	Glassmorphic dark UI; dynamic KPI metric cards; live sync heartbeat; auto-refresh countdown indicator.
5. Visualization Engine	Recharts, SVG Gradients	Renders cumulative timeline area charts, velocity bar charts with threshold reference lines, and category compositions.
6. Video Intelligence Dossier	Next.js Client Components	Embedded YouTube player preview, 4-bar Trend Score percentile inspector, and snapshot audit history with CSV download.

8. HARDWARE AND SOFTWARE SPECIFICATIONS

Software Requirements:

- Operating System: Windows 10/11, macOS, or Linux
- Runtime Environment: Node.js (v18.x or v20.x+)
- Framework: Next.js 14 (App Router)
- Language: TypeScript 5.5
- Database: PostgreSQL 18.x
- Styling: Tailwind CSS 3.4
- Charts: Recharts 2.12

Hardware Requirements:

- Processor: Dual-Core Intel/AMD Core i3 or higher
- Memory (RAM): Minimum 4 GB (8 GB recommended)
- Storage: 500 MB free disk space for runtime and DB
- Network: Active broadband connection for YouTube API polling

9. VERIFICATION & TESTING OUTCOMES

The platform underwent comprehensive verification across automated test scripts, integration checks, and UI browser tests:

- **API Authentication & Connectivity:** Verified against YouTube Data API v3; fetched 50 trending videos per country and 31 regional categories at only 2 quota units per region.

- **Database & Transaction Guard:** PostgreSQL local connection verified at 127.0.0.1:5432; duplicate snapshot guard validated via `ON CONFLICT (video_id, region, captured_at) DO NOTHING`.
- **REST API Endpoints:** Verified HTTP 200 responses across `/api/trending`, `/api/rising`, `/api/analytics`, and `/api/video/[id]`.
- **Build Integrity:** Production compilation (`npm run build`) validated with all 13 routes successfully pre-rendered. ESLint analysis passed with zero errors or warnings.
- **Interactive UI Validation:** Visual rendering of dark-mode glassmorphic theme, country selection pills, search filters, Recharts tooltips, and embedded playback confirmed via automated browser subagents.

10. CONCLUSION

The **YouTube Trend Intelligence Platform** successfully solves the problem of temporal opacity in video viewership. By establishing an append-only snapshot architecture paired with PostgreSQL window functions, the system derives accurate view velocity and transparent trend rankings while preserving free-tier quota limits. The application represents a complete, professional, full-stack software engineering project meeting all academic and industry standards for an MCA final-year capstone.